GALLIFY

Building Gallify, and deciding to wind it down

Gallify was an iOS app where digital artists could put up a gallery in a few minutes and collectors could hang the work on their own wall in AR before buying it. I joined early, became Chief Product and Operating Officer, and ran product and engineering until we closed the company.

ROLE
Co-founder, Chief Product/Operating Officer

TEAM
8 to 10 engineers in 4 sub-teams

STACK
iOS, ARKit, Object Capture, Solidity on Polygon

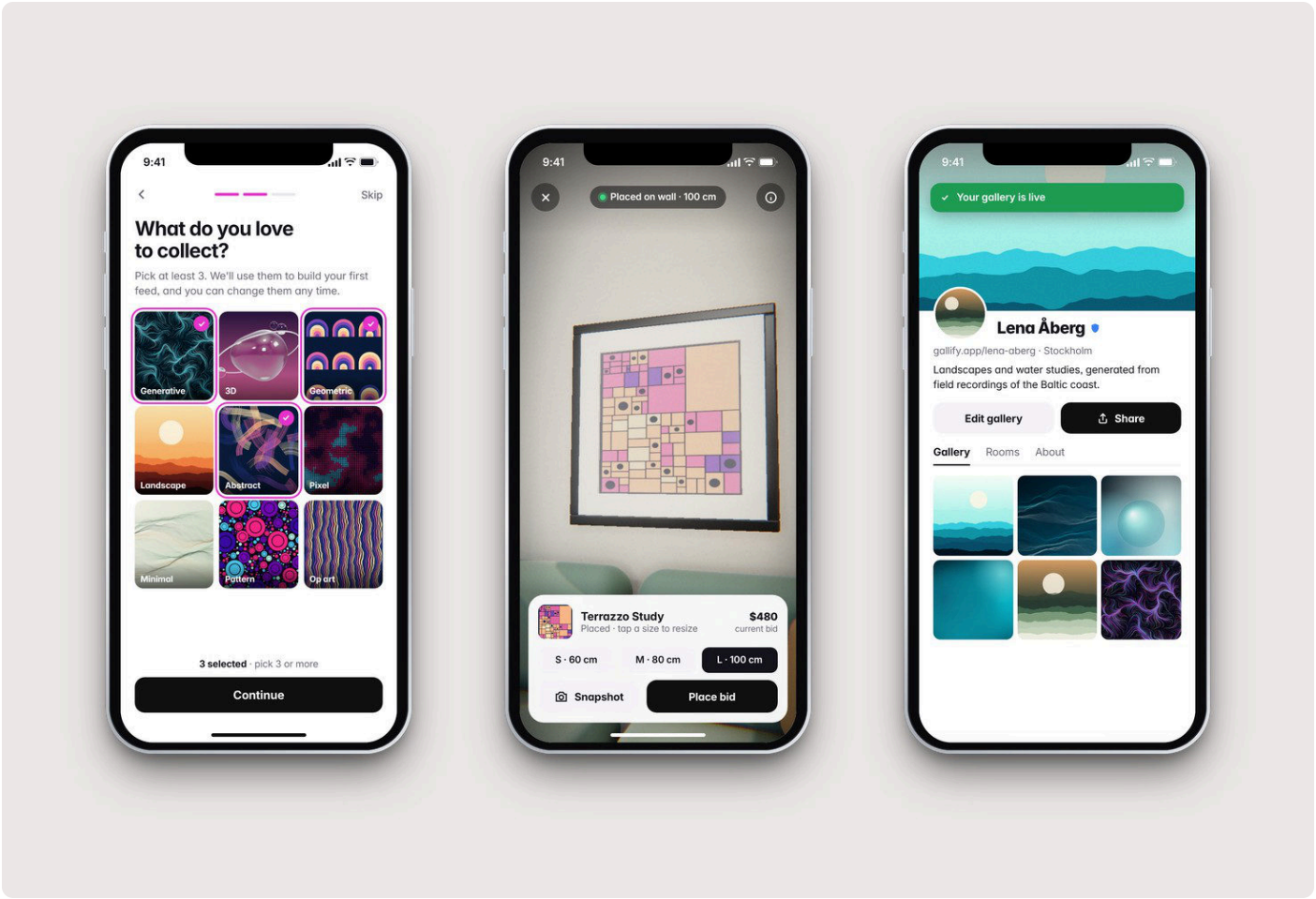
Code on GitHub **github.com/profio1997/Gallify_App**

+20%
click-through from recommendations

+10%
average order value

60→85%
on-time delivery

3 → 10+
engineers hired



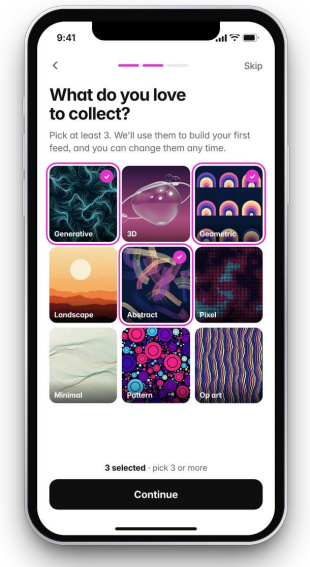
The polished screens on this page are recreations. Artist names and artwork are made up. The wireframes, AR capture and wallet screen are from the original build, shown in the same phone frame.

Overview

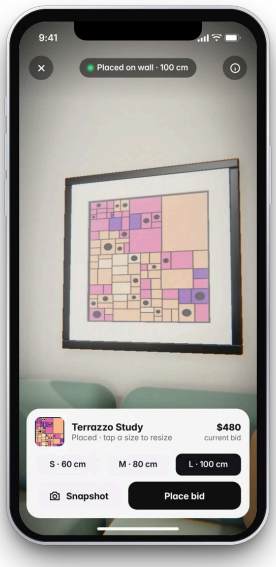
Gallify is where I learned the most about how to build a product and how to stop. It began with a small AI-focused team. I was hand-picked for my technical skills and for how I worked with others, and I grew into Chief Product and Operating Officer and a co-founder. Over five years I led product, ran engineering through a period of heavy hiring, and eventually recommended that we wind the company down.

The idea was simple to state. Digital artists had nowhere to show and sell their work that felt like a gallery, and collectors had no way to see a piece in their own space or to prove they owned it. We built an iOS app to fix both: galleries you could create and share in minutes, AR so you could see any piece on your wall at its real size, and ownership recorded on a blockchain.

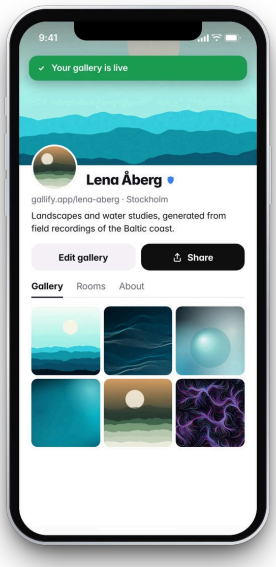
This page covers the product decisions, the recommender I led, how I ran the team, and the decision to stop.



Collector setup and discovery: pick a few styles, get a first feed built from them, then tap “Why?” to see and adjust what a pick is based on.



AR in a real room: walk around, place the piece on the wall at its real size, resize it, take a snapshot, then bid from the same screen.

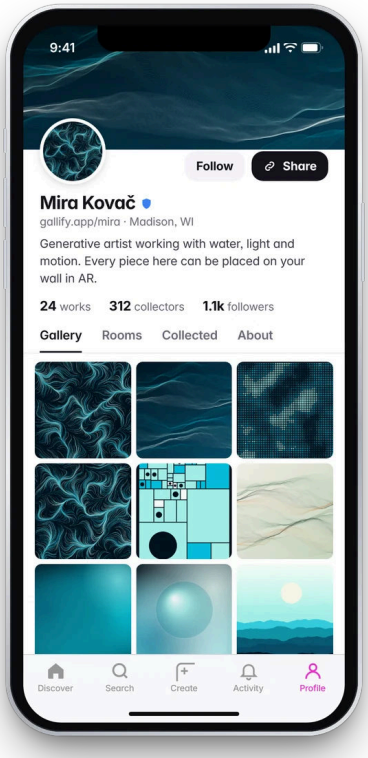


The artist side: add works, mint each as 1 of 1, publish a gallery and share its link.

Problem and research

Back then digital art mostly lived on social feeds built for photos. Artists could post, but there was no place that treated a piece of digital work with the presence a gallery gives a painting. For collectors there were two gaps. They couldn't tell how a piece would look at home, which is how most people decide to buy art, and they had no reliable way to prove they owned it, which is the thing that makes a collection worth anything.

Our positioning was "a next generation gallery for the digital age": create a gallery and share it instantly. We believed that seeing a piece on your own wall and being able to verify that you owned it would give digital art the weight it needed, and that good discovery would help collectors find work they'd never have searched for.



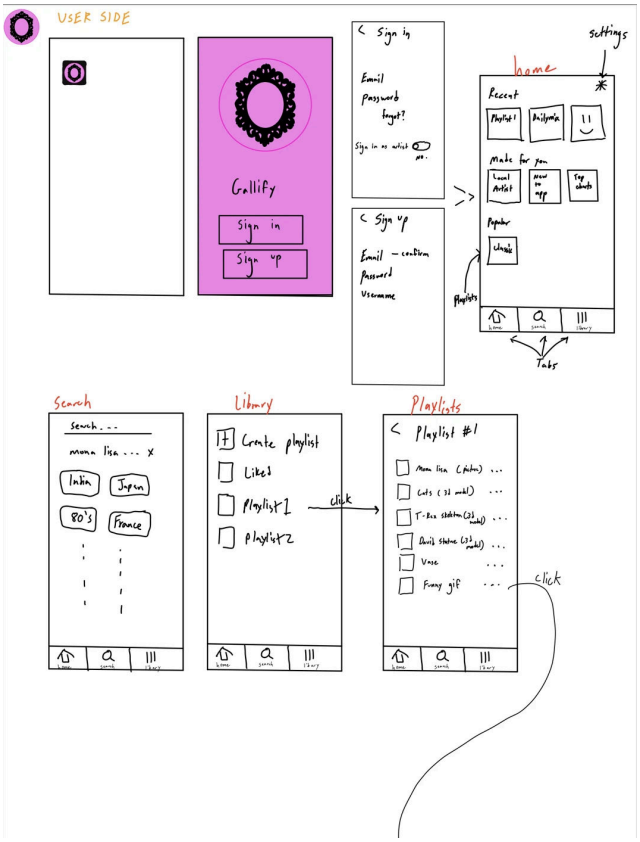
Each artist gets a gallery and a link to share it.

Discovery and early design

We began with flows, not screens. I sketched sign-up, the library, playlists and search by hand and argued over the paths until the team agreed on the journey. Only then did we move to high-fidelity iOS designs, and then to a beta landing page with a Discord community to collect early feedback.

The sketches show what mattered to us at the start: how an artist creates a gallery, how a collector builds a library of favourites, and how playlists let people share taste. A few of those ideas survived and a lot of them didn't, but starting on paper meant it cost us nothing to change our minds.

The beta page used the line "Create a gallery and share, instantly," and that sentence was our test of every feature: does it make the first gallery faster to create, or easier to share?



From the original build: early flow wireframes for sign-up, library, playlists and search.

Solution

Product decisions

Discovery over search
Collectors rarely know what to search for. So the home screen is a per-collector feed, with local auctions by city, curated playlists and a featured room, rather than a search bar.

AR as the differentiator
We built on Apple's ARKit so collectors could place a piece on their own wall at real size before bidding. Later we added Object Capture so artists could scan physical sculptures into 3D and show those in AR too.

Ownership on a blockchain
Purchases are tokens on Polygon (an ERC-721 contract written in Solidity) with wallet connect, so a purchase meant verifiable ownership. Polygon's low fees and fast confirmation suited small first purchases by new collectors.

Collectors keep their own keys
Bids and purchases are approved in the collector's own wallet. Gallify never holds anyone's keys, which meant fewer security obligations for us and more trust for them.

Lower the bar for artists
Publishing a gallery from a phone takes minutes: upload images or capture a 3D piece, mint the work as a one-of-one, and share one link.

How the pieces fit together

IOS APP
Auctions, discovery, galleries and profiles, built in Swift for iPhone

→

AR LAYER
ARKit places any piece on a wall at real size; Object Capture turns physical work into 3D

→

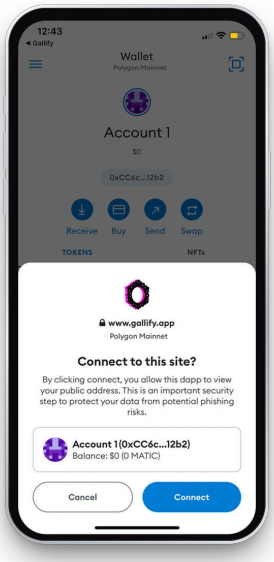
OWNERSHIP
Solidity smart contracts on Polygon issue each piece as a one-of-one token

→

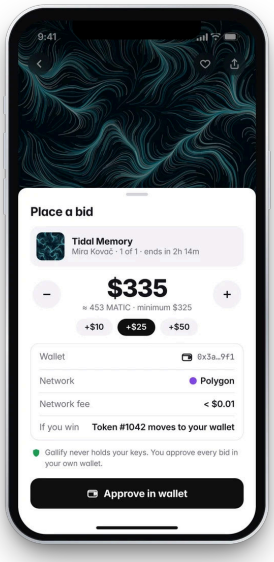
WALLET
The collector approves every bid and purchase in their own wallet



From the original build: a real AR capture in a bedroom.



From the original build: connecting a Polygon wallet.

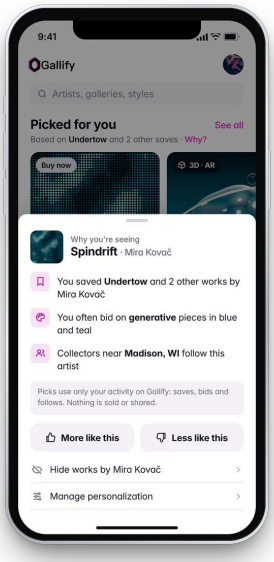


A bid, approved in the collector's own wallet.

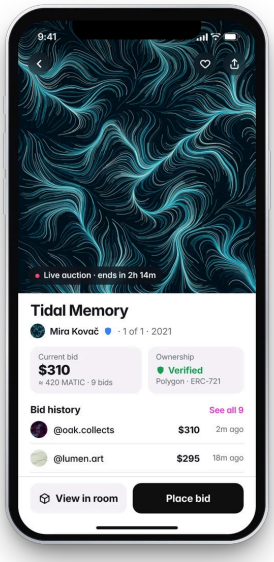
The recommender

I led the AI recommender for listings. Collectors arrived without a clear idea of what they wanted, and every artist wanted to be seen, so a feed that put the right piece in front of the right person was the most valuable thing we could build.

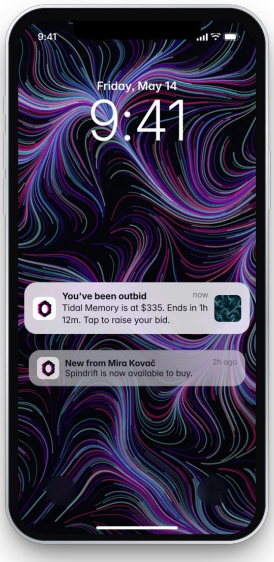
It ranked live auctions and listings for each collector from what they had viewed, saved, bid on and bought, plus each piece's style, medium and artist. New collectors picked a few styles they liked first so the feed wasn't empty on day one. Each pick had a reason behind it, and collectors could ask for more or less of something, or hide an artist. We measured the feed against our previous ordering, looking at click-through and at average order value rather than time in the app. Click-through rose 20% and average order value 10%.



Tapping "Why?" shows what a pick was based on and lets the collector adjust it.



A listing with live bidding and verified ownership.



An outbid alert while the auction is still running.

How the recommender worked

SIGNALS
Views, saves, bids and purchases; each piece's style, medium and artist; taste picks for new collectors

→

RANKING
Live auctions and listings, ordered for each collector and refreshed as they browse

→

EXPLANATION
A reason for each pick, with more, less and hide controls

→

MEASURED ON
Click-through and average order value, compared with the previous feed

Trade-offs I'd flag

Two of our biggest bets were also our riskiest, and each one cost us something.

AR. It was the feature people remembered, and it made the product feel like nothing else. It was also an engineering investment that only paid off if enough collectors cared about seeing a piece in their room. People loved the moment, but loving it and buying because of it are different things, and we never got clear evidence for the second.

Blockchain. On-chain ownership solved a real problem, which is proving that a piece is yours. It also added friction. New collectors had to set up a wallet before they could bid, and that is a lot to ask before someone has seen a single piece they love. We worked on this by keeping fees low on Polygon and by making the wallet step as clear as we could, but the cost to growth was real.

Team, results and the shutdown

Over five years we shipped three major releases. They covered marketplace transactions on the blockchain, the AR gallery experience and the iOS app itself, and we designed, built and launched the platform ahead of schedule. The recommender gave us the 20% and 10% lifts described above.

Running the team

By our busiest period we had six projects running across four sub-teams of eight to ten engineers. I was Scrum Master and also the main technical decision-maker, which meant setting product priorities and engineering standards. Delivery was unreliable when I took it on. Here's what I changed.

- **Rhythm.** I set up regular sprints and one-on-ones, so problems surfaced in a week instead of at the deadline.
- **Standards.** I wrote down engineering standards, including what "done" meant, so quality didn't depend on who was reviewing.
- **Hiring.** I ran the whole process, from sourcing to onboarding, as we grew from 3 engineers to more than 10, and recruited more than 15 people across the company.
- **Onboarding.** I documented the codebase and the way we worked so new engineers could contribute sooner.

DELIVERY MEASURE	RESULT
On-time delivery	**From about 60% to 85%**
Average bug resolution time	**About 35% faster**
Team velocity	**About 30% higher, with scope and budget within 10% of projections**
Time to productivity for new engineers	**About 25% shorter**
Active projects	**All 6 delivered on schedule, within 8% of estimates on average; production incidents under 2% of deployments**

Over five years we shipped three major releases. They covered marketplace transactions on the blockchain, the AR gallery experience and the iOS app itself, and we designed, built and launched the platform ahead of schedule. The recommender gave us the 20% and 10% lifts described above.

OUTCOME	RESULT
Click-through from recommendations	**+20%**
Average order value	**+10%**
Major releases	**3+: marketplace transactions, AR galleries, iOS**
Platform	**Designed, built and launched ahead of schedule**
Business	**Did not reach product-market fit; wound down**

Winding it down

The product worked. People used the AR view, and the recommendations measurably moved sales. What we never found was product-market fit. The group of collectors who wanted to buy digital art this way stayed small, and our delivery numbers, however good, didn't change that.

I recommended that we stop. It was the hardest call I've made, and I made it knowing that every month we kept going was a month the team put into a bet that wasn't paying off. Closing a company carefully (people, code and commitments) taught me nearly as much as building it did.

Before deciding I looked at four things: whether the people who tried Gallify came back, whether new collectors kept arriving without us pushing, whether we could afford another year of the same bet, and whether the team was still excited about it. None of the answers pointed toward continuing. I told the team directly, did what I could for the people who had put years into it, and made the app's code public so the work could still be seen. It's on GitHub today.

What I'd do differently

- We built AR and blockchain ownership before we had proof that enough collectors wanted them. Today I would test demand with something much cheaper first: a landing page, a handful of artists, a manual version of the experience.
- Our delivery numbers improved a lot, and they still told us nothing about product-market fit. I now decide up front which retention number would justify continuing, and I look at it every month.
- Stopping was the right decision. I would make it sooner.
- Telling users why a piece is shown, and letting them change it, was cheap to build and gave us a clearer signal than clicks alone. I'd do it from the start on any recommender.